

Source Code of App: Notification

admin.py

```
from django.contrib import admin

# Register your models here.
```

apps.py

```
from django.apps import AppConfig
from django.db.models.signals import post_migrate
from django.dispatch import receiver
import threading

class NotificationConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "Notification"

    def ready(self):
        from Notification import cron
        if not any(thread.name == "SchedulerThread" for thread in threading.enumerate()):
            print('Start')
            cron.start_scheduler()

    def on_migrate(self, sender, **kwargs):
        print("Database is ready after migration.")
```

cron.py

```
from apscheduler.schedulers.background import BackgroundScheduler
from django_apscheduler.jobstores import DjangoJobStore
from .models import NotificationSettings, FCMTToken
from .firebase_service import send_multiple_notifications
from django_apscheduler.models import DjangoJobExecution
from django.db import transaction
import threading
from datetime import datetime, timedelta

# สร้าง BackgroundScheduler
scheduler = BackgroundScheduler()
lock = threading.Lock()

def send_notifications(interval):
    """
    ฟังก์ชันที่รันตาม Scheduler เพื่อส่ง Notification
    Args:
        interval (int): ระยะเวลาการแจ้งเตือนเป็นนาที
    """
    with lock:
        try:
            print(f"Running notification job for interval: {interval} minutes")

            # ดึงเฉพาะผู้ใช้ที่มีการตั้งค่าตรงกับ interval ที่ระบุ
            settings_list = NotificationSettings.objects.filter(
                is_enabled=True,
                interval_minutes=interval
            ).select_related('user')

            tokens = []
            for settings in settings_list:
                try:
                    fcm_token = FCMTToken.objects.get(user=settings.user)
                    if fcm_token.token:
                        tokens.append(fcm_token.token)
                except FCMTToken.DoesNotExist:
                    print(f"ไม่มี FCM Token สำหรับผู้ใช้ {settings.user.username}")
                    continue

            if tokens:
                print(f"Sending notifications for {interval} minute interval to {len(tokens)} users")
                send_multiple_notifications(
                    tokens,
                    "การแจ้งเตือนออกกำลังกาย",
                    f"ถึงเวลาออกกำลังกายแล้ว! (แจ้งเตือนทุก {interval} นาที)"
                )
            else:
                print(f"No users to notify for {interval} minute interval")

        except Exception as e:
            print(f"Error in send_notifications for interval {interval}: {e}")

def create_notification_job(interval):
    """
    สร้าง job สำหรับการแจ้งเตือนที่ interval ที่กำหนด
    Args:
        interval (int): ระยะเวลาการแจ้งเตือนเป็นนาที
    """
    job_id = f'send_notifications_job_{interval}'
    scheduler.add_job(
        send_notifications,
        'interval',
        minutes=interval,
        id=job_id,
        args=[interval],
        replace_existing=True,
        next_run_time=datetime.now() # เริ่มทำงานทันที
    )
    print(f"Created notification job for {interval} minute interval with ID: {job_id}")

def start_scheduler():
    print("Starting scheduler...")
    try:
        if not scheduler.running:
            scheduler.add_jobstore(DjangoJobStore(), "default")

        # ลบ jobs เก่า
        for job in scheduler.get_jobs():
```

```

        scheduler.remove_job(job.id)
        print(f"Removed existing job: {job.id}")

# สร้าง jobs ใหม่
intervals = [1, 45, 60]
for interval in intervals:
    create_notification_job(interval)

scheduler.start()
print("Scheduler started successfully")
else:
    print("Scheduler is already running")
except Exception as e:
    print(f"Error starting scheduler: {e}")

```

firebase_service.py

```

from firebase_admin import credentials, messaging, initialize_app
import firebase_admin
import time

# Initialize Firebase
cred = credentials.Certificate(r"Notification\key\sit-to-fit-40890-firebase-adminsdk-tt616-ee3b9a7e1d.json")
firebase_admin.initialize_app(cred)
print("Firebase initialized")

def send_multiple_notifications(registration_tokens, title, body):
    print("Running scheduled task...")
    """
    ส่ง Push Notification ไปยังหลาย Token พร้อมกัน
    """
    if not registration_tokens:
        print("No registration tokens provided.")
        return None

    # แปลง single token เป็น list ถ้าจำเป็น
    if isinstance(registration_tokens, str):
        registration_tokens = [registration_tokens]

    success_count = 0
    failure_count = 0

    for token in registration_tokens:
        try:
            message = messaging.Message(
                notification=messaging.Notification(
                    title=title,
                    body=body
                ),
                webpush=messaging.WebpushConfig(
                    notification=messaging.WebpushNotification(
                        icon='/static/images/logo.jpg',
                        tag=str(int(time.time() * 1000)) # ใช้ timestamp เป็น tag
                    ),
                    headers={
                        'Urgency': 'high'
                    }
                ),
                data={
                    'timestamp': str(int(time.time() * 1000))
                },
                token=token
            )

            response = messaging.send(message)
            print(f"Successfully sent to token: {token}")
            success_count += 1

        except Exception as e:
            print(f"Failed to send to token {token}: {e}")
            failure_count += 1

    print(f"Completed sending notifications:")
    print(f"Success: {success_count}")
    print(f"Failed: {failure_count}")
    print(f"Total: {len(registration_tokens)}")

    return success_count > 0

```

forms.py

```

from django import forms
from .models import NotificationSettings

class NotificationSettingsForm(forms.ModelForm):
    class Meta:
        model = NotificationSettings
        fields = ['is_enabled', 'interval_minutes']

```

models.py

```

from django.db import models
from base.models import *
# Create your models here.
class FCMTToken(models.Model):
    user = models.OneToOneField(UserInformation, on_delete=models.CASCADE)
    token = models.CharField(max_length=255) # เก็บ FCM Token
    updated_at = models.DateTimeField(auto_now=True)

    def __str__(self):
        return f"{self.user.username} - {self.token}"

class NotificationSettings(models.Model):
    user = models.OneToOneField(UserInformation, on_delete=models.CASCADE)
    is_enabled = models.BooleanField(default=True) # เปิด/ปิดการแจ้งเตือน
    interval_minutes = models.IntegerField(
        choices=[(45, '45 นาที'), (60, '60 นาที'), (1, '1 นาที')],
        default=45
    )

```

```
def __str__(self):
    return f"Notification Settings for {self.user.username}"
```

send_notification.py

```
import firebase_admin
from firebase_admin import credentials, messaging

# เชื่อมยังไฟล์ service account ที่คุณดาวน์โหลดมา
cred = credentials.Certificate(r"key/sit-to-fit-40890-firebase-adminsdk-tt616-ee3b9a7e1d.json")
firebase_admin.initialize_app(cred)

def send_single_notification(registration_token, title, body):
    # สร้าง message object
    message = messaging.Message(
        notification=messaging.Notification(
            title=title,
            body=body
        ),
        token=registration_token,
    )

    # ส่ง message
    response = messaging.send(message)
    print("Successfully sent message:", response)

# ตัวอย่างการใช้งาน
my_token = "e01P7B1h2sIJmgElurlOi:APA91bF30QinQ7X2lJ32w193aEvubyuxRap_yyMnyrtoFwOJ8R1U40BTlLDiK7mBETHtdy_xAoLUoBYHegAsapbhohVkyAOq3utCE8_3gUI-1XMk9xxkikD4"
send_single_notification(my_token, "Hello!", "This is a test push notification")
```

signals.py

```
from django.db.models.signals import post_save
from django.dispatch import receiver
from django.db import transaction
from base.models import UserInfomation
from .models import NotificationSettings, FCMTOKEN

@receiver(post_save, sender=UserInfomation)
def create_notification_settings(sender, instance, created, **kwargs):
    """
    สร้างการตั้งค่าแจ้งเตือนเริ่มต้นสำหรับผู้ใช้งานใหม่
    """
    if created:
        with transaction.atomic():
            NotificationSettings.objects.get_or_create(
                user=instance,
                defaults={
                    'is_enabled': True,
                    'interval_minutes': 45 # ค่าเริ่มต้น
                }
            )
            FCMTOKEN.objects.get_or_create(
                user=instance,
                defaults={'token': ''} # สร้าง placeholder สำหรับ token
            )
```

urls.py

```
"""
URL configuration for SITTOFIT project.

The `urlpatterns` list routes URLs to views. For more information please see:
    https://docs.djangoproject.com/en/5.1/topics/http/urls/
Examples:
Function views
    1. Add an import:  from my_app import views
    2. Add a URL to urlpatterns:  path('', views.home, name='home')
Class-based views
    1. Add an import:  from other_app.views import Home
    2. Add a URL to urlpatterns:  path('', Home.as_view(), name='home')
Including another URLconf
    1. Import the include() function: from django.urls import include, path
    2. Add a URL to urlpatterns:  path('blog/', include('blog.urls'))
"""

from django.contrib import admin
from django.urls import path, include
from .views import notification_settings_view, save_fcm_token, app_to_pdf
app_name = 'Notification'

urlpatterns = [
    path('notification/', notification_settings_view, name='notification'),
    path('save-fcm-token/', save_fcm_token, name='save_fcm_token'),
    path('export-code/', app_to_pdf, name='export_code'),
]
```

views.py

```
from django.shortcuts import render, redirect
from django.contrib.auth.decorators import login_required
from .forms import NotificationSettingsForm
from .models import NotificationSettings, FCMTOKEN
from django.contrib import messages

from django.http import JsonResponse
from django.views.decorators.csrf import csrf_exempt
import json

@login_required
def notification_settings_view(request):
    settings, created = NotificationSettings.objects.get_or_create(user=request.user)

    if request.method == 'POST':
        form = NotificationSettingsForm(request.POST, instance=settings)
        if form.is_valid():
            form.save()
            messages.success(request, "การตั้งค่าการแจ้งเตือนถูกบันทึกเรียบร้อยแล้ว!")
```

```
        return redirect('Notification:notification')
    else:
        messages.error(request, "เกิดข้อผิดพลาด! กรุณาตรวจสอบข้อมูลที่กรอกและลองใหม่อีกครั้ง.")
    else:
        form = NotificationSettingsForm(instance=settings)

    return render(request, 'notification_settings.html', {'form': form, 'message': None})

@csrf_exempt
def save_fcm_token(request):
    if request.method == 'POST':
        data = json.loads(request.body)
        token = data.get('token')

        if request.user.is_authenticated:
            FCMToken.objects.update_or_create(
                user=request.user,
                defaults={'token': token}
            )
            return JsonResponse({'message': 'Token saved successfully!'})
        else:
            return JsonResponse({'error': 'User not authenticated'}, status=401)

    return JsonResponse({'error': 'Invalid request method'}, status=400)

import os
import pdfkit
from django.http import HttpResponse
from django.conf import settings

# กำหนด path ของ wkhtmltopdf (Windows ใช้ path นี้, Linux/Mac อาจไม่ต้อง)
PDFKIT_CONFIG = pdfkit.configuration(wkhtmltopdf=r"C:\Program Files\wkhtmltopdf\bin\wkhtmltopdf.exe\bin\wkhtmltopdf.exe")

def app_to_pdf(request, app_name):
    app_path = os.path.join(settings.BASE_DIR, app_name)
    pdf_filename = f"{app_name}.pdf"

    if not os.path.exists(app_path):
        return HttpResponse("App not found", status=404)

    html_content = f"
```

Source Code of App: {app_name}

```
"

    for root, dirs, files in os.walk(app_path):
        for file in files:
            if file.endswith((".py", ".html", ".css", ".js")): # แปลงเฉพาะไฟล์ที่ต้องการ
                file_path = os.path.join(root, file)
                with open(file_path, "r", encoding="utf-8") as f: # อ่านไฟล์เป็น UTF-8
                    html_content += f"

{file}

{f.read()}

"

# ใช้ pdfkit พร้อมกำหนดให้ใช้ encoding UTF-8
options = {'encoding': 'UTF-8'}
pdfkit.from_string(html_content, pdf_filename, configuration=PDFKIT_CONFIG, options=options)

with open(pdf_filename, "rb") as pdf:
    response = HttpResponse(pdf.read(), content_type="application/pdf")
    response["Content-Disposition"] = f'attachment; filename="{pdf_filename}"'
    return response
```

__init__.py

0001_initial.py

Generated by Django 5.1.2 on 2024-12-07 10:08

```
import django.db.models.deletion
from django.conf import settings
from django.db import migrations, models

class Migration(migrations.Migration):

    initial = True

    dependencies = [
        migrations.swappable_dependency(settings.AUTH_USER_MODEL),
    ]

    operations = [
        migrations.CreateModel(
            name="FCMToken",
            fields=[
                (
                    "id",
                    models.BigAutoField(
                        auto_created=True,
                        primary_key=True,
                        serialize=False,
                        verbose_name="ID",
                    ),
                ),
                ("token", models.CharField(max_length=255)),
                ("updated_at", models.DateTimeField(auto_now=True)),
                (
                    "user",
                    models.OneToOneField(
                        on_delete=django.db.models.deletion.CASCADE,
                        to=settings.AUTH_USER_MODEL,
                    ),
                ),
            ],
        ),
    ]
```

```
        ],
    ),
    migrations.CreateModel(
        name="NotificationSettings",
        fields=[
            (
                "id",
                models.BigAutoField(
                    auto_created=True,
                    primary_key=True,
                    serialize=False,
                    verbose_name="ID",
                ),
            ),
            ("is_enabled", models.BooleanField(default=True)),
            (
                "interval_minutes",
                models.IntegerField(
                    choices=[(45, "45 นาที"), (60, "60 นาที"), (1, "1 นาที)],
                    default=45,
                ),
            ),
            (
                "user",
                models.OneToOneField(
                    on_delete=django.db.models.deletion.CASCADE,
                    to=settings.AUTH_USER_MODEL,
                ),
            ),
        ],
    ),
],
),
]
```

0002_alter_notificationsettings_interval_minutes.py

Generated by Django 5.1.2 on 2024-12-07 15:55

```
from django.db import migrations, models

class Migration(migrations.Migration):

    dependencies = [
        ("Notification", "0001_initial"),
    ]

    operations = [
        migrations.AlterField(
            model_name="notificationsettings",
            name="interval_minutes",
            field=models.IntegerField(
                choices=[(45, "45 นาที"), (60, "60 นาที"), (1, "1 นาที)], default=45
            ),
        ),
    ]
```

__init__.py

firebase-messaging-sw.js

```
importScripts('https://www.gstatic.com/firebasejs/11.0.2/firebase-app-compat.js');
importScripts('https://www.gstatic.com/firebasejs/11.0.2/firebase-messaging-compat.js');

// ตั้งค่า Firebase
firebase.initializeApp({
  apiKey: "AIzaSyCmKxJerDOJc8hoQ7FrkA42RmjTwAg5B78",
  authDomain: "sit-to-fit-40890.firebaseio.com",
  projectId: "sit-to-fit-40890",
  storageBucket: "sit-to-fit-40890.firebaseioapp",
  messagingSenderId: "267449225965",
  appId: "1:267449225965:web:490cb8dcb99a97fdbdb2e4",
  measurementId: "G-YLTQJ1ZXPJ"
});
const messaging = firebase.messaging();

let lastNotificationId = null;
let lastNotificationTime = 0;
const NOTIFICATION_THRESHOLD = 1000; // 1 วินาที

// ฟังก์ชันที่ใช้ในการรับการแจ้งเตือน
messaging.onBackgroundMessage(function(payload) {
  console.log('Message received in SW:', payload);

  const currentTime = Date.now();
  if (currentTime - lastNotificationTime < NOTIFICATION_THRESHOLD) {
    console.log('Skipping duplicate notification');
    return;
  }

  lastNotificationTime = currentTime;

  const notificationOptions = {
    body: payload.notification.body,
    icon: '/static/images/logo.jpg',
    tag: currentTime.toString(),
    renotify: false,
    requireInteraction: false,
    silent: false,
    data: {
      notificationId: currentTime.toString()
    }
  };

  return self.registration.showNotification(
    payload.notification.title,
    notificationOptions
  );
});

// จัดการกับการคลิก notification
```

```
self.addEventListener('notificationclick', function(event) {
  console.log('Notification clicked:', event);
  event.notification.close();

  const urlToOpen = new URL('/', self.location.origin).href;

  const promiseChain = clients.matchAll({
    type: 'window',
    includeUncontrolled: true
  })
  .then((windowClients) => {
    for (let i = 0; i < windowClients.length; i++) {
      const client = windowClients[i];
      if (client.url === urlToOpen && 'focus' in client) {
        return client.focus();
      }
    }
    if (clients.openWindow) {
      return clients.openWindow(urlToOpen);
    }
  });

  event.waitUntil(promiseChain);
});
```

notification_settings.html

```
{% extends "base_noti.html" %}
{% block main %}
```

ตั้งค่าการแจ้งเตือน

```
{% if messages %}

  {% for message in messages %}

    {{ message }}

  {% endfor %}

{% endif %}

{% csrf_token %}
```

เปิดการแจ้งเตือน

```
{{ form.is_enabled }}
```

ช่วงเวลาการแจ้งเตือน (นาที)

```
{{ form.interval_minutes }}
```



```
{% if messages %}

{% endif %}
```

{% endblock %}